INSTITUTO FEDERAL
Brasília

Aula 4 – Aprofundando na programação do Arduino – Pt. 2

Marcos Santos
marcosvsantos@droidunb.com

INSTITUTO FEDERAL
Brasília

Aprofundamento ao if e else

if’s aninhados

Podemos **adicionar uma estrutura if ou if else**, dentro de uma **outra estrutura if, if else ou else já existente**. Isso pode se tornar muito útil quando temos **mais de uma condição** e elas seguem um **formato hierárquico**.

sketch_apr16a §

```
int temperatura = 30;
bool sensorAtivo = true;

void setup() {
  Serial.begin(9600);
}

void loop() {
  if (sensorAtivo == true) {
    if (temperatura > 30) {
      Serial.println("Alerta: Temperatura muito alta!");
    }
    else {
      Serial.println("Está tudo bem!");
    }
  }
  delay(1000);
}
```

Operadores lógicos

Operadores lógicos são muito úteis quando queremos **testar mais de uma expressão lógica em um único if**, tornando o código mais limpo e simples.

Utilizamos 3 operadores lógicos em C:

- **&& (and)**
- **|| (or)**
- **! (not)**

sketch_apr16a §

```
int temperatura = 30;
bool sensorAtivo = true;

void setup() {
  Serial.begin(9600);
}

void loop() {
  if (sensorAtivo == true && (temperatura > 40 || temperatura < 5)) {
    Serial.println("Alerta: Temperatura extrema!");
  }
  delay(1000);
}
```

INSTITUTO FEDERAL
Brasília

Estrutura switch e case

switch e case

A estrutura switch e case é muito utilizada quando se quer **realizar diferentes ações para diferentes valores que uma variável do tipo int ou char pode ter**. Portanto, é necessário especificar a variável testada no switch e especificar cada caso com suas respectivas ações.

sketch_apr16a §

```
int modo = 2;

void setup() {
  Serial.begin(9600);
}

void loop() {
  switch (modo) {
    case 1:
      Serial.println("Modo Econômico");
      break;
    case 2:
      Serial.println("Modo Desempenho");
      break;
    default:
      Serial.println("Modo Padrão");
      break;
  }
  delay(1000);
}
```

switch e case

Ao rodar o programa, **será verificado se o valor da variável entra no caso, para cada caso** na ordem em que foram escritos. **Se entrar, faz as respectivas ações**. Se não tiver um **“break”**, faz todas as ações dos outros casos automaticamente. Se houver, a estrutura chega ao fim.

sketch_apr16a §

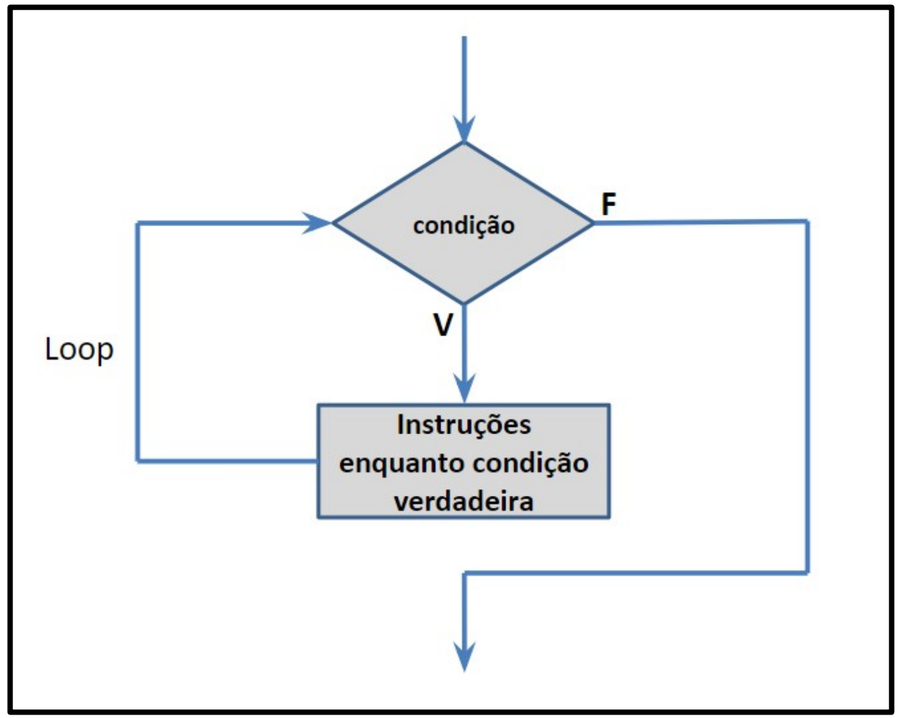
```
int modo = 2;

void setup() {
  Serial.begin(9600);
}

void loop() {
  switch (modo) {
    case 1:
      Serial.println("Modo Econômico");
      break;
    case 2:
      Serial.println("Modo Desempenho");
      break;
    default:
      Serial.println("Modo Padrão");
      break;
  }
  delay(1000);
}
```

INSTITUTO FEDERAL
Brasilía

Loop’s de repetição

O que são?

Loop's de repetição **são uma forma de fazer um bloco de ações enquanto uma expressão lógica tiver um valor**. Esses loop's podem rodar uma quantidade padrão de vezes ou mesmo "eternamente", tudo depende do algoritmo de seu código.



while

O loop while **roda o bloco de ações quando a expressão lógica passada como parâmetro é true**, se for **false ele finaliza o loop** e volta para o código principal. Se colocarmos somente **true ou 1 como parâmetro**, o loop **roda "eternamente"**.

sketch_apr16a §

```
void setup() {
  pinMode(A0, INPUT);
  Serial.begin(9600);
}

void loop() {
  int sensor = digitalRead(A0);

  while(sensor == 1) {
    Serial.println("linha vista, siga ela");
  }
  while(sensor == 0) {
    Serial.println("não há linha, volte no trajeto");
  }
}
```

for

No C, a **única diferença** entre o loop for e while é a **necessidade de declarar ao menos uma variável** de qualquer tipo que tenha uma relação com números e **devemos dizer as modificações que serão feitas cada vez que esse loop rodar**.

sketch_apr16a §

```
void setup() {
  pinMode(A0, INPUT);
  Serial.begin(9600);
}

void loop() {
  int sensor = digitalRead(A0);

  for(int i = 3; sensor == 1 && i >= 0; i--){
    if(i == 0)
      Serial.println("Corre");
    else
      Serial.println(i);
  }
}
```

for

Os parâmetros de modificação nas variáveis seguem a seguinte escrita **("i" é a variável criada no for** nesses exemplos, e **n é sempre um número qualquer**):

- **i += n** (incrementa n)
- **i -= n** (decrementa n)
- **i *= n** (multiplica por n)
- **i /= n** (divide por n)
- **i++** (incrementa 1 a i)
- **i--** (decrementa 1 de i)

sketch_apr16a §

```
void setup() {
  pinMode(A0, INPUT);
  Serial.begin(9600);
}

void loop() {
  int sensor = digitalRead(A0);

  for(int i = 3; sensor == 1 && i >= 0; i--){
    if(i == 0)
      Serial.println("Corre");
    else
      Serial.println(i);
  }
}
```

for

O for **também precisa de uma expressão lógica** que comandará quantas vezes o loop será executado, **assim como o while**, com exceção de que essa expressão pode envolver aquela(s) variável(eis) criada(s) nos parâmetros do for.

sketch_apr16a §

```
void setup() {
  pinMode(A0, INPUT);
  Serial.begin(9600);
}

void loop() {
  int sensor = digitalRead(A0);

  for(int i = 3; sensor == 1 && i >= 0; i--){
    if(i == 0)
      Serial.println("Corre");
    else
      Serial.println(i);
  }
}
```

INSTITUTO FEDERAL
Brasília

Contadores no Arduino

Contadores no Arduino

Contadores vem da necessidade de **contar quantas vezes algo rodou, ou quantas vezes obtivemos certo valor de um sensor**, ou ideias parecidas. Dessa forma, o contadores são **muito utilizados em conjunto à laços de repetição**. Eles podem oferecer uma boa **sincronização das tarefas** e **noções simples de tempo**. Veja um exemplo do mesmo contador em uma versão usando while e outra usando for.

sketch_apr16a §

```
void setup() {
  Serial.begin(9600);

  int i = 0;
  while(i < 100) {
    Serial.println(i);
    i++;
  }

  for(int i = 0; i<100; i++) {
    Serial.println(i);
  }
}

void loop() {}
```

INSTITUTO FEDERAL
Brasília

Exercícios

Exercício 1

Uma variável possui informações da temperatura, em graus, de uma localidade do Brasil. Com isso, faça um código que verifica essa variável e printa a seguinte mensagem para os seguintes intervalos:

- acima de 35: “muito quente”;
- entre 35 e 26: “quente”;
- entre 26 e 22: “normal”;
- entre 22 e 16: “frio”;
- menor que 16: “muito frio”.

Exercício 2

Uma variável possui informações da nota de um estudante, sendo um valor inteiro entre 0 e 10. Com isso, faça um código que verifica essa nota e printa uma mensagem específica, de sua escolha, para ela. Ou seja, teremos 10 possibilidades de mensagens únicas.

Exercício 3

Faça um código com dois contadores, um que incrementa do 0 até o número 100 e outro que decrementa do número 100 até o 0. Printe o novo valor do contador a cada mudança que ele sofrer.

Exercício 4

Crie três variáveis com valores inteiros, cada um representando uma reta com tamanho igual aos seus respectivos valores. Faça um código que analisa se é possível montar um triângulo com essas retas. Se sim, o código printa “isso pode virar triangulo”, se não, printa “eh impossivel virar triangulo”.

Exercício 5

Faça um código que receba um número inteiro de 0 a 9 do Monitor Serial e faça o LED interno do arduino piscar a quantidade de vezes recebida.

Utilize esse bloco de código para receber o número inteiro de 0 a 9 digitado no monitor serial:

```
if(Serial.available()) {
  int n = Serial.read();
}
```

INSTITUTO FEDERAL
Brasilía

Parabéns pelo empenho!

Até a próxima aula!